

Replay / May 2026

Durable Agents

Long-running AI workflows in a flaky world

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**
- Founder & CEO of **Pydantic Labs**
- Working on **Pydantic AI**, **Logfire**, and **Monty**
- Writing Python and Rust developer
- Now AI talking head (like everyone else)

The Pydantic stack

- **Pydantic Validation** - contracts for data
- **Pydantic AI** - production grade agent framework, **with durable execution**
- **Pydantic Logfire** - observability, evals, prompt management, LLMOps
- **Pydantic AI Gateway** - one endpoint, every model
- **Pydantic Monty** - Sandboxed Python interpreter

AI is still Engineering

AI doesn't run in a vacuum, isolated from the practical challenges of the cloud.

The advent of AI makes good engineering practices more important, not less.

Reliable AI



An agent

Structured, type-safe output with Pydantic AI.

```
from datetime import date, datetime
from pydantic import BaseModel
from pydantic_ai import Agent

class ConferenceTalk(BaseModel):
    title: str
    start_time: datetime
    room: str | None = None

conference_talk_agent = Agent(
    "gateway/anthropic:claude-haiku-4-5",
    output_type=list[ConferenceTalk],
    instructions="Extract information about conference talks",
)
```

Make the agent observable

Two lines (and an import) to make the agent observable.

```
import logfire

logfire.configure()
logfire.instrument_pydantic_ai()

...
```

Trace Observability

Trace · 5.50s · 9 spans

● live filters · search

POST /support/ticket/45af-3...	http		5.50s
└─SELECT * FROM tickets WHE...	pg		42ms
└─support_agent run	pydantic-ai		5.39s
└─ chat claude-haiku-4-5	pydantic-ai		1.32s
└─ running tool: lookup_or...	pydantic-ai		964μs
└─ chat claude-haiku-4-5	pydantic-ai		1.92s
└─ running tool: query_shi...	pydantic-ai		1.20ms
└─ chat claude-haiku-4-5	pydantic-ai		2.14s
└─ result.output="Great ne...	logfire		—

support_agent run

Run Details Raw

▶ INPUT system

You are a support agent for an online retailer. Use the available tools to look up orders and shipments before responding. ... show more

▶ INPUT user

My order #4521 is missing — can you tell me where it is?

▶ OUTPUT assistant

I'll look up your order and check the shipment status for you.

Tool call · lookup_order

```
{
  "order_id": 4521
}
```

▶ OUTPUT tool lookup_order

```
{
  "order_id": 4521,
  "customer": "jean@example.com",
  "shipped_on": "2026-04-26",
  "carrier": "DHL",
  "tracking": "JD0123456789"
}
```

What happens when it runs for hours?

Make the agent durable

One line (and an import) to make the agent durable.

```
...
```

```
from pydantic_ai.durable_exec.temporal import TemporalAgent
```

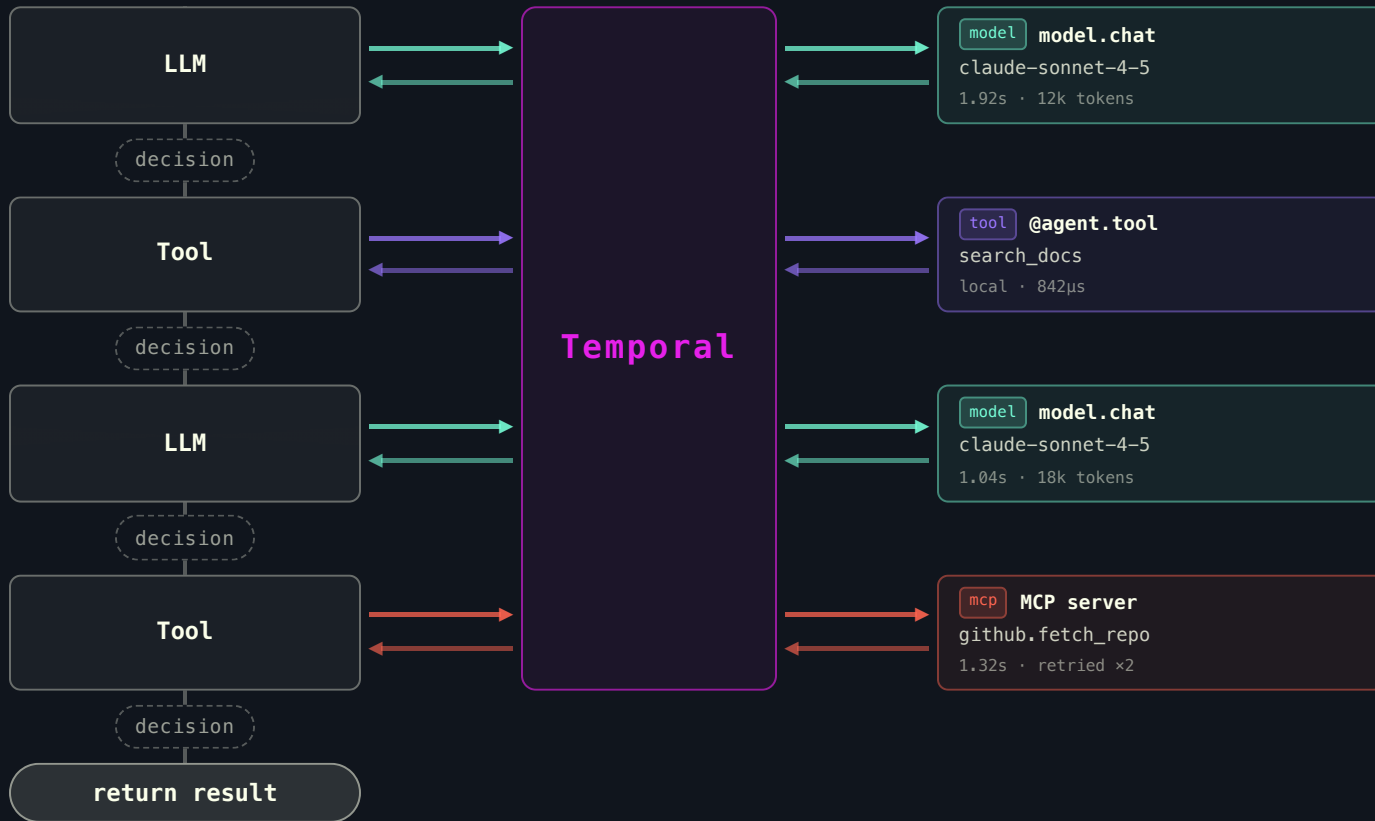
```
durable_conference_talk_agent = TemporalAgent(conference_talk_agent)
```

No need to manually wrap your tool calls as activities.

How this works

- The **agent loop** lives in the workflow - completely deterministic
- **Every external call** becomes an activity:
 - model requests
 - `@agent.tool` functions
 - MCP server calls
- One `TemporalAgent` wraps your agent and wires it all up
- If the worker dies mid-run, Temporal replays the workflow and resumes - without re-paying for the model calls already done.

How this works



What this gets you

- **Resume mid-run** - agents resume after a worker dies - from where it left off
- **Automatic retries** - flaky tools, rate-limited models, etc
- **Long-running by default** - `workflow.sleep(timedelta(days=7))` just works
- **No bespoke orchestration** - no queues, no checkpoint tables, no state machines you have to write yourself

Demo



Thank you

Slides, examples and docs link:



github.com/pydantic/talks/tree/main/2026-05-replay